TUTORIAL: PWM

Timer Output Compare

Name: HWANG MooYeol ID: 22000814

I. Introduction

In this lab, you will learn how to set up MCU Timers for Timer Interrupt and PWM output.

Objectives of this lab are learning how to

- Configure registers of Timers (TIMx)
- Generate PWM signals

Hardware

NUCLEO -F411RE

Software

Keil uVision IDE, CMSIS, EC_HAL

Documentation

STM32 Reference Manual

II. Basics of Timer Interrupt

A. Register List

TIMx registers

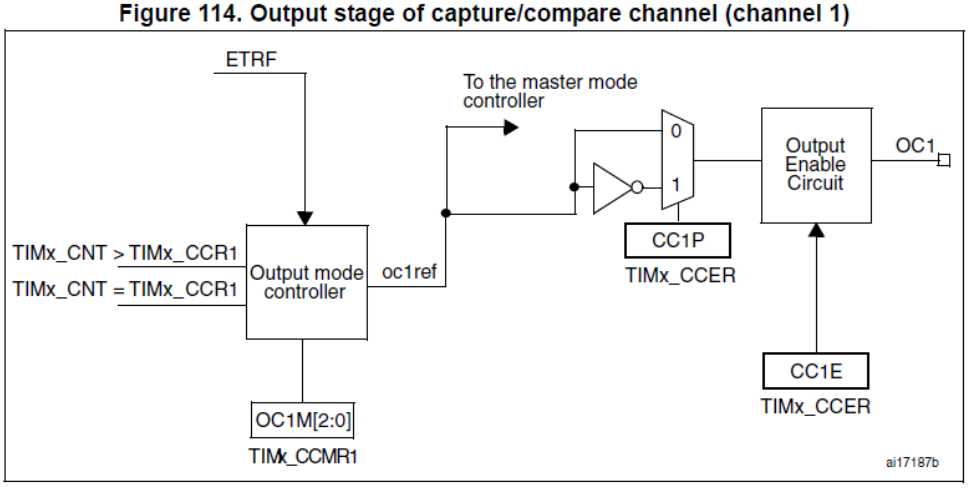
Type	Register Name	Description
TIMx	TIMx_ CR1	TIMx control register 1
	TIMx_ PSC	TIMx prescaler register
	TIMx_ARR	TIMx auto-reload register
	TIMx_DIER	TIMx DMA Interrupt Enable register
	TIMx_BDTR	TIM1(only) break and dead-time register

PWM output registers

Type	Register Name	Description
TIMx	TIMx _CCMRy	TIMx capture/compare mode register for yth channel
	TIMx _CCRy	TIMx capture/compare register for yth channel
	TIMx _CCER	TIMx capture/compare output enable register

Schematic

Figure 114. Output stage of capture/compare channel (channel 1)



B. Register Setting

System Clock setting

1. RCC setting (PLL)

GPIO Pin setting

1. Set RCC for GPIO

2. AF(TIMx) mode selection for Pin_y in GPIOx

Timer setting:

1. Enable Timer peripheral Clock (**RCC→APB1ENR**)

2. Set Timer clock Pre-scaler value (**TIMx→PSC : PSC[15:0]**)

3. Set Auto-reload value (**TIMx→ARR : ARR**)

4. Set Counting direction (**TIMx→CR1 : DIR**)

5. Enable Counter (**TIMx→CR1 : CEN**)

PWM Out setting:

1. Set PWM Output mode (**TIMx→CCMR : OCyM**)

2. Set CompareCapture value (**TIMx→CCRy : CCR**)

3. Select Output Polarity (**TIMx→CCER : CCyP**)

4. Enable CompareCaptureOutput (**TIMx→CCER : CCyE**)

III. Tutorial

A. Register Configuration

Fill in the table

Register	Description	Register setting
RCC	PLL Initialization	RCC_PLL_init(); EC_SYS_CLK = EC_PLL = 84,000,000
	Enable Timer Peripheral Clock: TIM2	RCC->APB1ENR |=1≪0
TIM2 (Timer setting)	TIM2 counting direction: DIR0	TIM2->CR1 & = ~ (1≪4)
	Set Timer Clock Pre-scaler value 84MHz To 100kHz	TIM2->PSC = 0b1101000111 // 839
	Set Auto-reload value: With 100kHz, counting of 1kHz	TIM2->ARR = 0b1100011 //99
	Enable Timer DMA/Interrupt	TIM2 -> DIER |= 1<<0
	Enable Counter	TIM2->CR1 |= 1<<0
GPIOA	Set GPIOA pin 5 as Alternate Function mode: 10	GPIOA->MODER &=~(0b11<<2*5) GPIOA->MODER |= (0b10<<2*5)

	GPIOA Alternate Function Selection: AF1(TIM2/TMI5) AFRL_5[3:0]=0001	GPIOA->AFR[0] &= ~(0b1111<<4 *5) GPIOA->AFR[0] |= (0b0001<<4 *5)
	AF Output as No-PUPD, Push-Pull,Very High Speed.	// write your HAL API Port-> PUPDR |= (0<<5*2) Port -> OTYPER |= (0<<5) Port -> OSPEEDR |= (3<<5*2)
TIM2 (PWM setting)	Output compare mode as PWM mode1	TIM2->CCMR1 &= ~(0b111 << 4) TIM2->CCMR1 |= (0b110 << 4)
	Enable preload on CCR1	TIM2->CCMR1 |= (0b1 << 3)
	Set CCR value for 50% duty ratio: (ARR+1)/2	TIM2->CCR1 = 50
	Output as active high	TIM2->CCER &= ~(1<<1)
	Enable Compare and Capture output	TIM2->CCER |= 1

B. Programming

Procedure

- Create a new folder '**EC/Tutorial/TU_Timer_PWM/**'
- Open the program 'Keil uVision5' and create a new project.
- Name the project as '**TU_Timer_PWM'**.
- Create a new item called '**TU_Timer_PWM.c**'
- Use the given source code of '**TU_Timer_PWM_student.c**' Click to download

- Fill in the empty spaces in the code.
- Run the program and check your result.
- Your tutorial report must be submitted to LMS

Exercise

Output PWM on TIM2_CH1 (PA5): 1kHz with Dutry-ratio of 0 / 50% / 100%

- System CLK is PLL 84MHz for STM32F411RE
- TIM2: Up-counting, Timer2_CLK = 100 kHz, COUNT_CLK=1 kHz
- PWM Mode 1 / Period 1kHz / Duty ratio 50%.
- PWM Out on PA_5 (TIM2_CH1)
- There are several pins for TIM2_CH1. We will use GPIO Pin5 (LD2) for tutorial
- Set the GPIO pin as Alternate function (AF) for TIM2

  : No pull-up & No pull-down / High speed / Push-Pull

  : AF on PA_5 (TIM2_CH1)

- Give PWM output of 0%, 50%, 100% of 1kHz PWM
- Check the brightness of LD2 (PA5) for each different Duty ratio

```
/**
*****************************************************************************
* @author   HWANG MooYeol
* @Mod            2024-8-23 by Jin Kwak
```

```
* @brief     Embedded Controller:   Tutorial PWM
*
*
******************************************************************************
*/
#include "stm32f411xe.h"
#include "ecGPIO2_student.h"
#include "ecRCC2.h"
#include "ecSysTick2.h"

#define LED_PIN      PA_5

void setup(void);

int main(void) {
        // Initialiization -------------------------------------------------------
        RCC_PLL_init();                                 // System Clock = 84MHz
        SysTick_init();                                 // for delay_ms()
        GPIO_init(LED_PIN, AF);        // GPIOA 5 ALTERNATE function
        GPIO_ospeed(LED_PIN, HIGH_SPEED);     // GPIOA 5 HIGH SPEED

        // TEMP: TIMER Register Initialiization ------------------------------------------------
------
        TIM_TypeDef *TIMx;
        TIMx = TIM2;

        // GPIO: ALTERNATIVE function setting
        GPIOA->AFR[0] &= ~(0b1111<<4 *LED_PIN);         // Clear
        GPIOA->AFR[0] |= (0b0001<<4 * LED_PIN);        // AF1 at PA5 = TIM2_CH1
(p.150)
```

```
        // TIMER: PWM setting

        RCC->APB1ENR |=   1;                                    // Enable TIMER
clock



        TIMx->CR1 &=   ~(1<<4);
        // Direction Up-count



        TIMx->PSC =   0b1101000111;    //839                                  // Set
Timer CLK = 100kHz : (PSC + 1) = 84MHz/100kHz --> PSC = ?



        TIMx->ARR =    0b1100011;                        //99
                       // Auto-reload: Upcounting (0...ARR).


                                                                                //
Set Counter CLK = 1kHz : (ARR + 1) = 100kHz/1kHz --> ARR = ?



        TIMx->CCMR1 &= ~TIM_CCMR1_OC1M;                            // Clear
ouput compare mode bits for channel 1

        TIMx->CCMR1 |=   0b110 << 4;                               // OC1M =
110 for PWM Mode 1 output on ch1

        TIMx->CCMR1  |= TIM_CCMR1_OC1PE;                    // Output 1 preload
enable (make CCR1 value changable)



        TIMx->CCR1 =      50;
                //Duty ratio 50%          // Output Compare Register for channel 1



        TIMx->CCER &= ~TIM_CCER_CC1P;                              // select
output polarity: active high

        TIMx->CCER |=           1;
                                        // Enable output for ch1



        TIMx->CR1   |= TIM_CR1_CEN;                         // Enable counter



        // Inifinite Loop -------------------------------------------------------

        while(1){
```

```
                                //Create the code to change the brightness of LED as 10kHZ
(use "delay(1000)")
                        delay_ms(1000);
                        }
            }
// Initialiization
void setup(void)
{
            RCC_PLL_init();           // System Clock = 84MHz
            SysTick_init();           // for delay_ms()
}
```

Appendix

[See here for MCU resources]

Timer PinOut Map

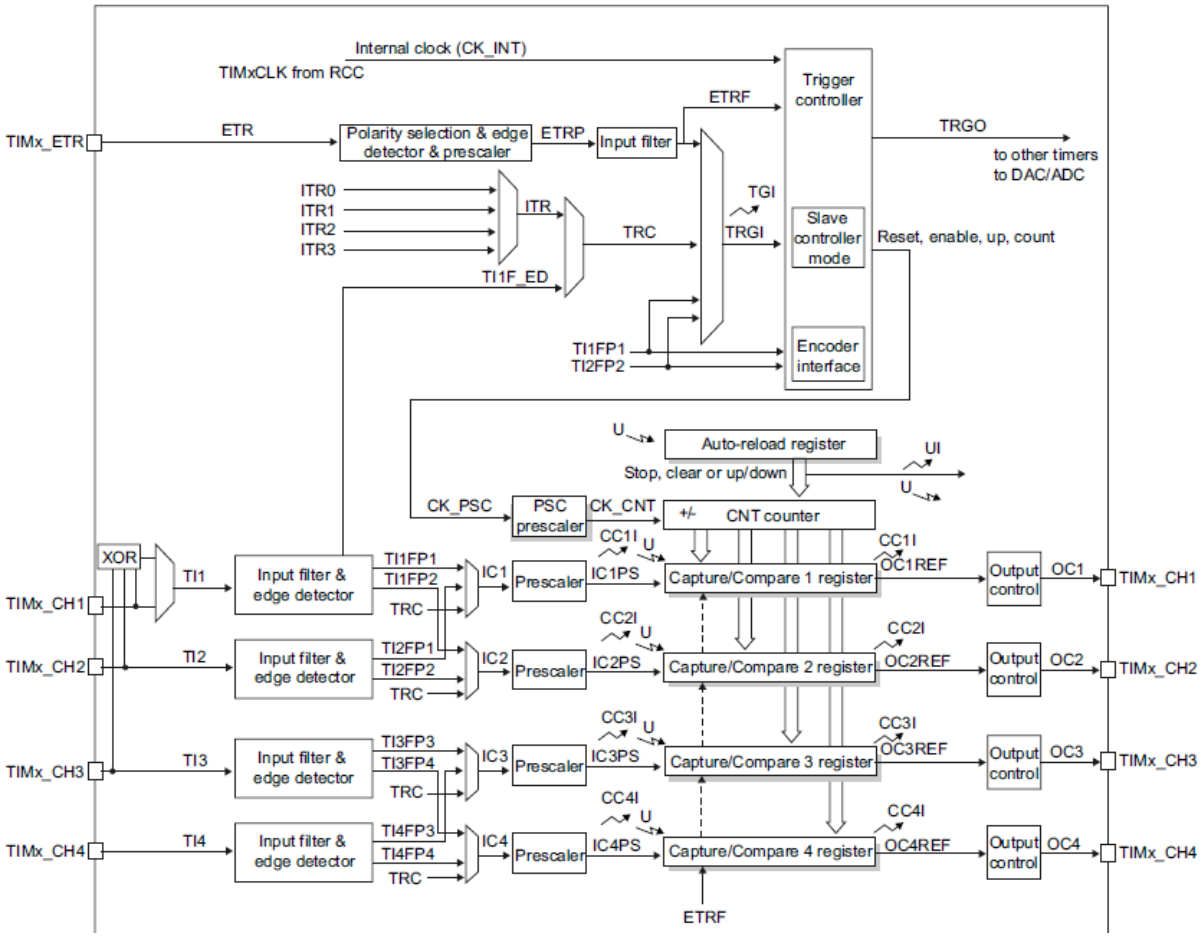
Advanced Timer

Timer	Channel	Port	Pin
1	1	A	8
	1N	A	7
		B	13
	2	A	9
	2N	B	0
		B	14
	3	A	10
	3N	B	1
		B	15
	4		
	4N		

General Purpose Timer

Timer	Channel	Port	Pin
2	1	A	0
		A	5
		A	15
	2	A	1
		B	3
	3	B	10
	4		
3	1	A	6
		B	4
		C	6
	2	B	5
		C	7
	3	C	8
	4	C	9
4	1	B	6
	2	B	7
	3	B	8
	4	B	9

Embedded Controller



Embedded Controller

- TIM_TypeDef : <stm32f411xe.h>

```
typedef struct
{
  __IO uint32_t CR1;         /*!< TIM control register 1,              Address offset: 0x00 */
  __IO uint32_t CR2;         /*!< TIM control register 2,              Address offset: 0x04 */
  __IO uint32_t SMCR;        /*!< TIM slave mode control register,     Address offset: 0x08 */
  __IO uint32_t DIER;        /*!< TIM DMA/interrupt enable register,   Address offset: 0x0C */
  __IO uint32_t SR;          /*!< TIM status register,                 Address offset: 0x10 */
  __IO uint32_t EGR;         /*!< TIM event generation register,       Address offset: 0x14 */
  __IO uint32_t CCMR1;       /*!< TIM capture/compare mode register 1, Address offset: 0x18 */
  __IO uint32_t CCMR2;       /*!< TIM capture/compare mode register 2, Address offset: 0x1C */
  __IO uint32_t CCER;        /*!< TIM capture/compare enable register, Address offset: 0x20 */
  __IO uint32_t CNT;         /*!< TIM counter register,                Address offset: 0x24 */
  __IO uint32_t PSC;         /*!< TIM prescaler,                       Address offset: 0x28 */
  __IO uint32_t ARR;         /*!< TIM auto-reload register,            Address offset: 0x2C */
  __IO uint32_t RCR;         /*!< TIM repetition counter register,     Address offset: 0x30 */
  __IO uint32_t CCR1;        /*!< TIM capture/compare register 1,      Address offset: 0x34 */
  __IO uint32_t CCR2;        /*!< TIM capture/compare register 2,      Address offset: 0x38 */
  __IO uint32_t CCR3;        /*!< TIM capture/compare register 3,      Address offset: 0x3C */
  __IO uint32_t CCR4;        /*!< TIM capture/compare register 4,      Address offset: 0x40 */
  __IO uint32_t BDTR;        /*!< TIM break and dead-time register,    Address offset: 0x44 */
  __IO uint32_t DCR;         /*!< TIM DMA control register,            Address offset: 0x48 */
  __IO uint32_t DMAR;        /*!< TIM DMA address for full transfer,   Address offset: 0x4C */
  __IO uint32_t OR;          /*!< TIM option register,                 Address offset: 0x50 */
} TIM_TypeDef;
```

```
#define TIM2                ((TIM_TypeDef *) TIM2_BASE)
#define TIM3                ((TIM_TypeDef *) TIM3_BASE)
#define TIM4                ((TIM_TypeDef *) TIM4_BASE)
#define TIM5                ((TIM_TypeDef *) TIM5_BASE)
```

```
/*!< APB1 peripherals */
#define TIM2_BASE           (APB1PERIPH_BASE + 0x0000UL)
#define TIM3_BASE           (APB1PERIPH_BASE + 0x0400UL)
#define TIM4_BASE           (APB1PERIPH_BASE + 0x0800UL)
#define TIM5_BASE           (APB1PERIPH_BASE + 0x0C00UL)
```